

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSE 4101 / [Course Code]

PROJECT REPORT

Project Title:

Online Medicine Shop

Semester: [Spring 2025–2026]

Submission Date: [18 May 2026]

Section: [W]

Submitted To:

MIRZA ASIF MAHMUD

[Lecturer, Department of Computer Science]
American International University-Bangladesh

Group Information

Group Number: 05

Project Title: Online Medicine Shop

Student Information:

Full Name: Sheikh Tahmina Moshtari Chandni

Student ID: 23-51027-1

Program: Bsc Cse

Email: 23-51027-1@student.aiub.edu

Signature: _____

Student Information:

Full Name: Abu Jhaed Md Rakib

Student ID: 23-51026-1

Program: Bcs Cse

Email: 23-51026-1@student.aiub.edu

Signature: _____

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.		Section	
Member 1 ID			

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			Feature Implementation (0-3)		Completeness (0-3)
Basic Web Security	UI (HTML/CSS)	Feature Completeness	DB	Auth (Session/cookie)	MVC
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			Collaborative Teamwork (0-3)	Communication/Promptness (0-3)	
JS validation	PHP Validation	Ajax/JSON	Git Contribution	VIVA	
Expected Outcome (0-2)		CO3 Total	CO4 Total		Project Total

Evaluated By: _____ Date: _____

Course Teacher Signature: _____

Table of Contents

1. Group Information	2
2. Marks Sheet	3
3. Table of Contents	4
4. Chapter 1: Project Overview	9
4.1 Project Description	9
4.1 Technologies Used	10
4.3 System Architecture / MVC Diagram	11
4.4 Database Schema	12
5. Chapter 2: Git Contribution & Task Allocation	13
5.1 GitHub Repository Link	13
5.2 Git Contribution Screenshot	14
6. Chapter 3: Features, Source Code & Screenshots	16
7. Chapter 5: Conclusion & Future Work	18

Chapter 1: Project Overview

1.1 Project Description

Online Medicine Shop is a web portal that gives the Customer facility of easy purchase of medicines from anywhere at any time and completely control over the products, orders, and users of admin users and also solving the common problem of access to life saving medicines at the time of critical situation, without going from the corner shop to the many shops. And ensuring the user friendly and efficient way of browsing, ordering and order approval.

Target users are twofold:

Customers Registered users can look for medicines either by searching for name, genre (e.g. Aspirin genre, Paracetamol genre) or type (liquid or solid form). Users can add medicines to car, update the amount, and at the end can make checkout.

They can specify shipping address and method of paying (via system or cash), get order confirmation. Admin – people authorized to manage Medicine inventory (full CRUD) categories delete Customer accounts, view all Purchase requests, accept/reject Orders and see complete purchase history of all customers.

Centralized platform addresses the problem of time-consuming manual medicine ordering process by providing a role-based platform capable of providing inventory records, secured authentication, and performing order tracking which ensures that medicines are on time.

1.2 Technologies Used

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3, Bootstrap	UI & Styling
Frontend Scripting	JavaScript / jQuery	Client-side validation, AJAX
Backend	PHP	Server-side logic
Database	MySQL	Data storage]
Auth	PHP Sessions / Cookies	Login management
Version Control	Git / GitHub	Collaboration & versioning
Server	XAMPP / Apache	Local development

1.3 MVC Diagram

CONTROLLER				
Controller	Auth • AuthController.php	Home / Profile • HomeController.php • ProfileController.php	Medicine / Category • MedicineController.php • CategoryController.php	Cart / Order / Admin • CartController.php • OrderController.php • AdminController.php
Actions	Search • search.php	Cart actions • cart_add.php • cart_update.php • cart_remove.php	Order status • order_status.php	Entry: index.php Assets: css/ js/ uploads/

MODEL				
Model	Database • Database.php • (PDO connection)	User / Category • User.php • Category.php	Medicine / Cart • Medicine.php • Cart.php	Order / Payment • Order.php • Payment.php

VIEW				
View	Auth • login.php • register.php	Medicines • medicine_list.php • medicine_create.php • medicine_edit.php	Categories • category_list.php • category_create.php	Shared layout • header.php • footer.php • home.php • profile.php
	Cart / Checkout • cart.php • checkout.php	Order • invoice.php • success.php	Admin • dashboard.php • customers.php • purchase_requests.php • history.php	↑ HTML Response to Browser

1.4 Database Schema.

Table Name	Field Name	Data Type	Description
Table: users			
users	id	INT	Primary Key, Auto Increment
	email	VARCHAR(100)	Unique, login credential
	role	ENUM	admin or customer
	password_hash	VARCHAR(255)	Hashed password
Table: categories			
categories	id	INT	Primary Key, Auto Increment
	name	VARCHAR(100)	Category name
	category_type	ENUM	liquid or solid
Table: medicines			
medicines	id	INT	Primary Key, Auto Increment
	category_id	INT	Foreign Key → categories(id)
	price	DECIMAL(10,2)	Medicine price
	availability	INT	Stock quantity
Table: cart			
cart	id	INT	Primary Key, Auto Increment
	user_id	INT	Foreign Key → users(id)
	medicine_id	INT	Foreign Key → medicines(id)
	quantity	INT	Quantity added to cart
Table: orders			
orders	id	INT	Primary Key, Auto Increment
	user_id	INT	Foreign Key → users(id)
	total_amount	DECIMAL(10,2)	Total order amount
	status	ENUM	pending, accepted, rejected
Table: order_items			
order_items	id	INT	Primary Key, Auto Increment
	order_id	INT	Foreign Key → orders(id)
	medicine_id	INT	Foreign Key → medicines(id)
	quantity	INT	Quantity ordered

Table Name	Field Name	Data Type	Description
	unit_price	DECIMAL(10,2)	Price at time of order
Table: payments			
payments	id	INT	Primary Key, Auto Increment
	order_id	INT	Foreign Key → orders(id)
	transaction_id	VARCHAR(100)	Unique payment reference

Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

Repository URL: https://github.com/sheikhchandni55/medicine_store

2.2 Git Contribution Screenshot

```
User@DESKTOP-UC1CQR5 MINGW64 ~/OneDrive/Desktop
$ git clone https://github.com/sheikhchandni55/medicine_store.git
fatal: destination path 'medicine_store' already exists and is not an empty directory.

User@DESKTOP-UC1CQR5 MINGW64 ~/OneDrive/Desktop
$ cd medicine_store

User@DESKTOP-UC1CQR5 MINGW64 ~/OneDrive/Desktop/medicine_store (main)
$ |
```

Chapter 3: Features, Source Code & Screenshots

Feature 1: User Authentication (Login & Registration)

Description:

This feature manages user authentication in the system. It allows users to register, log in, and log out securely. It also includes a “Remember Me” option that keeps users logged in using cookies. This ensures secure access control and improves user convenience.

Relevant Source Code

File: medicine_shop/controller/AuthController.php

```
// Authentication Controller
class AuthController extends BaseController {

    private $userModel;

    // Constructor
    function __construct() {
        $this->userModel = new User();
    }

    // ===== LOGIN =====
    function login() {

        // Redirect if already logged in
        if ($this->isLoggedIn()) {
            $this->redirect('controller=home&action=index');
        }

        // Handle form submission
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {

            // Find user by email
            $user = $this->userModel->findByEmail($_POST['email']);

            // Verify password
            if ($user && password_verify($_POST['password'], $user['password_hash'])) {

                // Store session data
                $_SESSION['user_id'] = $user['id'];
                $_SESSION['name'] = $user['name'];
                $_SESSION['role'] = $user['role'];

                // Remember Me option
                if (isset($_POST['remember'])) {
                    $token = bin2hex(random_bytes(32));
                    $this->userModel->updateRememberToken($user['id'], $token);
                    setcookie('remember_token', $token, time() + (86400 * 30), '/');
                }

                // Redirect to home
                $this->redirect('controller=home&action=index');
            }
        } else {
            // Invalid credentials
            $this->view('login', ['error' => 'Invalid credentials']);
        }
    } else {
        // Load login page
        $this->view('login');
    }
}
```

```
// ===== REGISTER =====
function register() {

    if ($this->isLoggedIn()) {
        $this->redirect('controller=home&action=index');
    }

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {

        $errors = [];

        // Validation
        if (strlen($_POST['password']) < 8)
            $errors[] = 'Password must be 8+ chars';

        if ($_POST['password'] !== $_POST['confirm_password'])
            $errors[] = 'Passwords do not match';

        if ($this->userModel->findByEmail($_POST['email']))
            $errors[] = 'Email already exists';

        // Create user
        if (empty($errors)) {
            $this->userModel->create($_POST);
            $this->redirect('controller=auth&action=login');
        } else {
            $this->view('register', ['errors' => $errors]);
        }
    } else {
        $this->view('register');
    }
}
```

```
// ===== LOGOUT =====
function logout() {

    // Remove remember token
    if (isset($_SESSION['user_id'])) {
        $this->userModel->updateRememberToken($_SESSION['user_id'], null);
    }

    // Destroy session & cookie
    setcookie('remember_token', '', time() - 3600, '/');
    session_destroy();

    // Redirect to login
    $this->redirect('controller=auth&action=login');
}

// ===== REMEMBER ME =====
function checkRememberMe() {

    if (!$this->isLoggedIn() && isset($_COOKIE['remember_token'])) {

        // Find user by token
        $user = $this->userModel->findByRememberToken($_COOKIE['remember_token']);

        // Restore session
        if ($user) {
            $_SESSION['user_id'] = $user['id'];
            $_SESSION['name'] = $user['name'];
            $_SESSION['role'] = $user['role'];
        }
    }
}
```

Code Explanation:

- `$_SERVER['REQUEST_METHOD'] === 'POST'` → Checks if form is submitted
- `$_POST['email'], $_POST['password']` → Gets user input data
- `findByEmail()` → Fetches user from database using email
- `password_verify()` → Compares input password with hashed password
- `$_SESSION['user_id'], name, role` → Stores user login session
- `setcookie()` → Saves “Remember Me” token in browser
- `redirect()` → Sends user to home page
- Else → Shows error or reloads login/register page

Screenshot:

The image displays two screenshots of the MediShop web application interface. Both screenshots feature a dark blue header with the MediShop logo and navigation links for Home, Login, and Register.

The top screenshot shows the "Create Account" form. It includes the following fields and elements:

- Full Name:** A text input field containing "Sheikh Chandni".
- Email:** A text input field containing "chandnisheikh163@gmail.com".
- Password (min 8 characters):** A text input field with masked characters (dots).
- Confirm Password:** A text input field with masked characters (dots).
- Role:** A dropdown menu with "Admin" selected.
- Address:** A text input field containing "Dhaka, Bangladesh".
- Phone:** A text input field containing "01743832455".
- Register:** A blue button at the bottom of the form.

The bottom screenshot shows the "Login to MediShop" form. It includes the following fields and elements:

- Email:** A text input field containing "chandnisheikh163@gmail.com".
- Password:** A text input field with masked characters (dots).
- Remember Me:** A checkbox that is checked.
- Login:** A blue button.
- Don't have an account? [Register here](#)**: A link to the registration page.

Feature 2: Medicine Management (CRUD)

Description:

The MedicineController manages all medicine-related operations in the system using MVC structure. It allows only admin users to access its features. The controller can display all medicines, add new medicine with image upload, edit existing medicine details, and delete medicines from the system. It also includes input validation to ensure correct data (name, price, stock, image type/size) before saving to the database. Additionally, it handles secure image uploading by generating unique filenames and storing them in the uploads folder.

Relevant Source Code

File: medical_shop/controller/MedicineController

```
<?php

class MedicineController extends BaseController {

    private $medicineModel;

    public function __construct() {
        $this->requireAdmin(); // Only admin access
        $this->medicineModel = new Medicine();
    }

    // ===== LIST =====
    public function index() {
        $medicines = $this->medicineModel->getAll();
        $this->view('medicine_list', ['medicines' => $medicines]);
    }

    // ===== CREATE =====
    public function create() {

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {

            $errors = $this->validateMedicine($_POST, $_FILES);

            if (empty($errors)) {

                $imagePath = null;
                if ($_FILES['image']['error'] === UPLOAD_ERR_OK) {
                    $imagePath = $this->uploadImage($_FILES['image']);
                }

                $data = [
                    'name' => $_POST['name'],
                    'category_id' => $_POST['category_id'],
                    'vendor_name' => $_POST['vendor_name'],
                    'price' => $_POST['price'],
                    'availability' => $_POST['availability'],
                    'description' => $_POST['description'],
                    'image_path' => $imagePath
                ];

                $this->medicineModel->create($data);
                $this->redirect('controller=medicine&action=index');
                return;
            }

            $this->view('medicine_create', ['errors' => $errors, 'old' => $_POST]);

        } else {
            $this->view('medicine_create');
        }
    }
}
```

```
// ===== EDIT =====
public function edit() {

    $id = $_GET['id'];
    $medicine = $this->medicineModel->findById($id);

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {

        $data = [
            'name' => $_POST['name'],
            'category_id' => $_POST['category_id'],
            'vendor_name' => $_POST['vendor_name'],
            'price' => $_POST['price'],
            'availability' => $_POST['availability'],
            'description' => $_POST['description']
        ];

        if ($_FILES['image']['error'] === UPLOAD_ERR_OK) {
            $data['image_path'] = $this->uploadImage($_FILES['image']);
        }

        $this->medicineModel->update($id, $data);
        $this->redirect('controller=medicine&action=index');
        return;
    }

    $this->view('medicine_edit', ['medicine' => $medicine]);
}

// ===== DELETE =====
public function delete() {
    $id = $_GET['id'];
    $this->medicineModel->delete($id);
    $this->redirect('controller=medicine&action=index');
}

// ===== VALIDATION =====
private function validateMedicine($data, $files) {
    $errors = [];

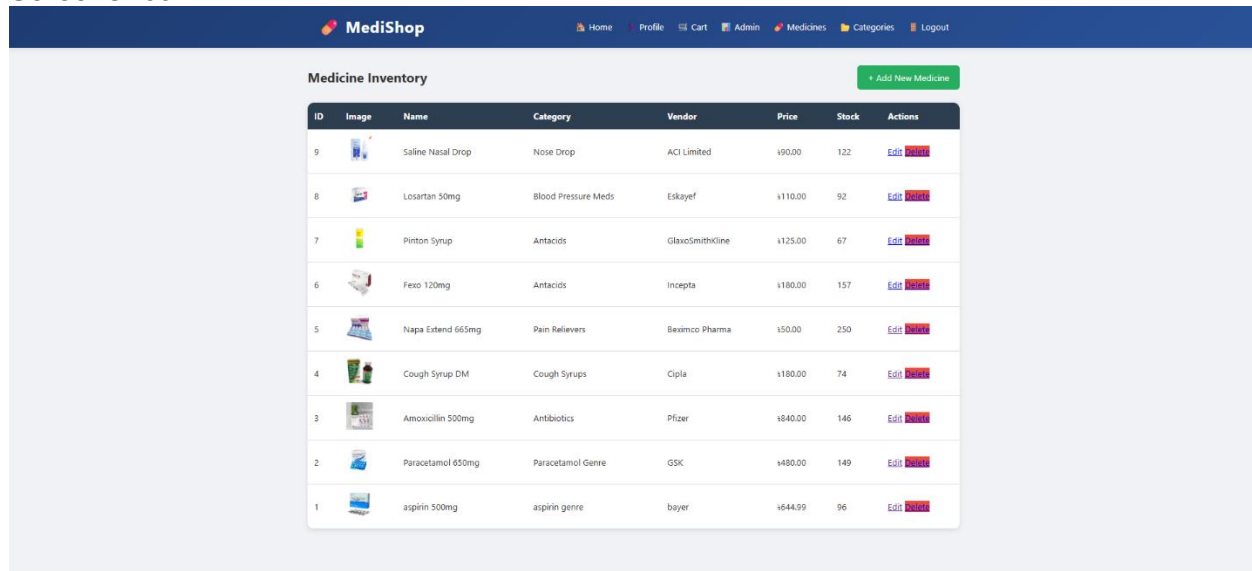
    if (empty($data['name'])) $errors[] = "Name required";
    if ($data['price'] <= 0) $errors[] = "Valid price required";
    if (!isset($files['image'])) $errors[] = "Image required";

    return $errors;
}

// ===== IMAGE UPLOAD =====
private function uploadImage($file) {
    $filename = 'med_' . time() . '.jpg';
    $path = __DIR__ . '/../uploads/medicines/' . $filename;
    move_uploaded_file($file['tmp_name'], $path);
    return $filename;
}
```

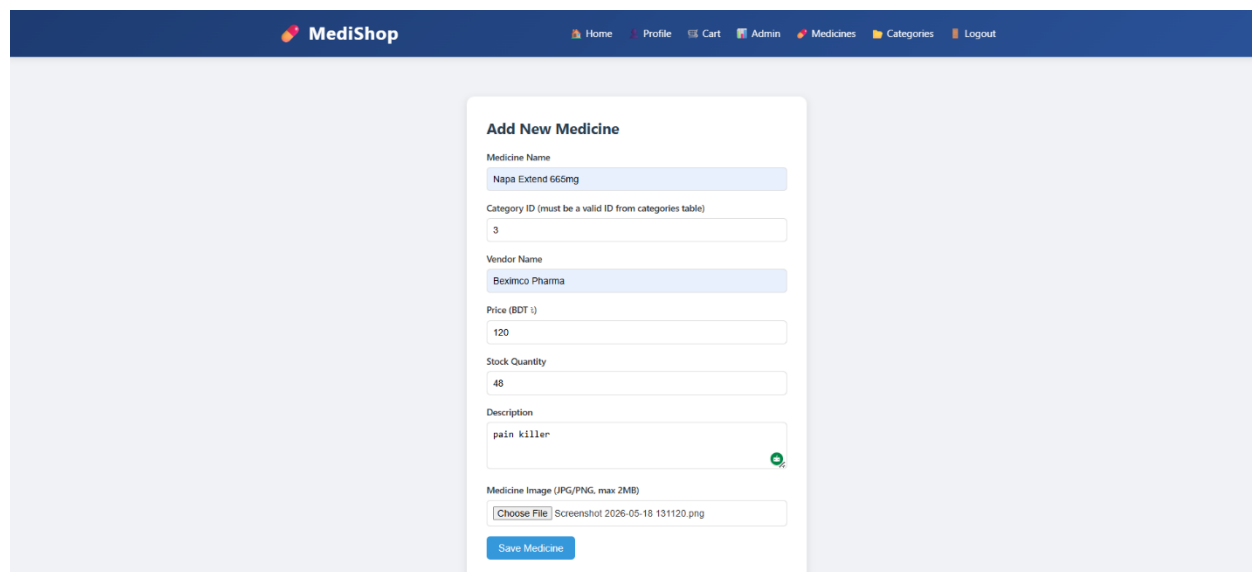
Code explanation:

- ❑ requireAdmin() → Only admin can access medicine management
- ❑ index() → Fetches all medicines and shows list page
- ❑ \$_SERVER['REQUEST_METHOD'] === 'POST' → Checks form submission
- ❑ create() → Validates input, uploads image, saves new medicine
- ❑ edit() → Loads medicine by ID and updates its data
- ❑ delete() → Removes medicine from database by ID
- ❑ validateMedicine() → Checks required fields, price, and image rules
- ❑ uploadImage() → Generates unique filename and stores image in folder

Screenshot:


The screenshot shows the 'Medicine Inventory' page of the MediShop application. It features a table with 9 columns: ID, Image, Name, Category, Vendor, Price, Stock, and Actions. There are 9 rows of medicine data. A green '+ Add New Medicine' button is located at the top right of the table.

ID	Image	Name	Category	Vendor	Price	Stock	Actions
9		Saline Nasal Drop	Nose Drop	ACI Limited	₳90.00	122	Edit Delete
8		Losartan 50mg	Blood Pressure Meds	Eskayef	₳110.00	92	Edit Delete
7		Pinton Syrup	Antacids	GlaxoSmithKline	₳125.00	67	Edit Delete
6		Fexo 120mg	Antacids	Incepta	₳180.00	157	Edit Delete
5		Napa Extend 665mg	Pain Relievers	Beximco Pharma	₳50.00	250	Edit Delete
4		Cough Syrup DM	Cough Syrups	Cipla	₳180.00	74	Edit Delete
3		Amoxicillin 500mg	Antibiotics	Pfizer	₳80.00	146	Edit Delete
2		Paracetamol 650mg	Paracetamol Genre	GSK	₳80.00	149	Edit Delete
1		aspirin 500mg	aspirin genre	bayer	₳44.99	96	Edit Delete



The screenshot shows the 'Add New Medicine' form in the MediShop application. The form contains the following fields and values:

- Medicine Name:** Napa Extend 665mg
- Category ID (must be a valid ID from categories table):** 3
- Vendor Name:** Beximco Pharma
- Price (BDT.):** 120
- Stock Quantity:** 48
- Description:** pain killer
- Medicine Image (JPG/PNG, max 2MB):** Choose File | Screenshot 2026-05-18 131120.png

A green checkmark icon is visible next to the description field, indicating it is valid. A blue 'Save Medicine' button is at the bottom of the form.

Feature 3: Category Management**Description:**

The CategoryController manages all category-related operations in the system using the MVC pattern. It is restricted to admin users only using `requireAdmin()`. The controller allows viewing all categories (index), adding new categories (create), updating existing categories (edit), and deleting categories (delete). It interacts with the Category model to perform database operations. If a category cannot be deleted due to linked medicines, it stores an error message in the session and redirects back to the category list page.

Relevant Source Code

File: medical_shop/controller/CategoryController

```
<?php
class CategoryController extends BaseController {
    private $categoryModel;

    public function __construct() {
        $this->requireAdmin(); // Admin only access
        $this->categoryModel = new Category();
    }

    // ===== LIST =====
    public function index() {
        $categories = $this->categoryModel->getAll();
        $this->view('category_list', ['categories' => $categories]);
    }

    // ===== CREATE =====
    public function create() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $this->categoryModel->create($_POST['name'], $_POST['category_type']);
            $this->redirect('controller=category&action=index');
        } else {
            $this->view('category_create');
        }
    }

    // ===== EDIT =====
    public function edit() {
        $id = $_GET['id'];

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $this->categoryModel->update($id, $_POST['name'], $_POST['category_type']);
            $this->redirect('controller=category&action=index');
        } else {
            $category = $this->categoryModel->findById($id);
            $this->view('category_create', ['category' => $category]);
        }
    }

    // ===== DELETE =====
    public function delete() {
        $id = $_GET['id'];

        if (!$this->categoryModel->delete($id)) {
            $_SESSION['error'] = 'Cannot delete category with medicines';
        }

        $this->redirect('controller=category&action=index');
    }
}
```

Code Explanation

- `requireAdmin()` → Only admin can access category management
- `index()` → Fetches all categories and displays list
- `create()` → Adds new category from form input
- `edit()` → Updates category data by ID
- `delete()` → Deletes category, shows error if linked medicines exist
- `$_SERVER['REQUEST_METHOD']` → Checks form submission
- `$_GET['id']` → Gets category ID from URL
- `redirect()` → Redirects back to category list page

Screenshot:

New Category

Category Name

Type

[Save](#)

Categories [+ Add Category](#)

ID	Name	Type	Actions
6	Antacids	solid	Edit Delete
7	Antacids	liquid	Edit Delete
3	Antibiotics	solid	Edit Delete
1	aspirin genre	solid	Edit Delete
8	Blood Pressure Meds	solid	Edit Delete
4	Cough Syrups	liquid	Edit Delete
9	Nose Drop	liquid	Edit Delete
5	Pain Relievers	solid	Edit Delete
2	Paracetamol Genre	solid	Edit Delete

Feature 4: Shopping Cart System**Description:**

The CartController manages all shopping cart operations for logged-in users in the system. It ensures only authenticated users can access cart features using `requireLogin()`. The controller allows users to view their cart (`index`), add medicines to cart (`addAjax`) after checking stock availability, update item quantities (`updateAjax`), and remove items (`removeAjax`). It also provides real-time cart count updates (`getCountAjax`) using AJAX responses. The controller calculates total price dynamically based on product price and quantity and communicates with Cart and Medicine models to handle database operations securely.

Relevant Source Code

File: `medical_shop/controller/CartController`

```
<?php
class CartController extends BaseController {

    private $cartModel;
    private $medicineModel;

    public function __construct() {
        $this->requireLogin(); // user must be logged in
        $this->cartModel = new Cart();
        $this->medicineModel = new Medicine();
    }

    // ===== SHOW CART =====
    public function index() {
        $cartItems = $this->cartModel->getUserCart($_SESSION['user_id']);

        $total = array_sum(array_map(
            fn($i) => $i['price'] * $i['quantity'],
            $cartItems
        ));

        $this->view('cart', [
            'cartItems' => $cartItems,
            'total' => $total
        ]);
    }

    // ===== ADD TO CART (AJAX) =====
    public function addAjax() {

        $medicineId = $_POST['medicine_id'] ?? 0;
        $quantity = (int)($_POST['quantity'] ?? 1);

        if (!$this->medicineModel->checkStock($medicineId, $quantity)) {
            $this->json(['success' => false, 'error' => 'Insufficient stock']);
            return;
        }

        $result = $this->cartModel->addItem($_SESSION['user_id'], $medicineId, $quantity);
        $count = $this->cartModel->getCartCount($_SESSION['user_id']);

        $this->json(['success' => $result, 'cart_count' => $count]);
    }
}
```

```
// ===== UPDATE CART (AJAX) =====
public function updateAjax() {

    $cartId = $_POST['cart_id'];
    $quantity = (int)$_POST['quantity'];

    $result = $this->cartModel->updateQuantity($cartId, $quantity);

    $cartItems = $this->cartModel->getUserCart($_SESSION['user_id']);

    $total = array_sum(array_map(
        fn($i) => $i['price'] * $i['quantity'],
        $cartItems
    ));

    $this->json(['success' => $result, 'total' => $total]);
}

// ===== REMOVE FROM CART (AJAX) =====
public function removeAjax() {

    $cartId = $_POST['cart_id'];

    $result = $this->cartModel->removeItem($cartId);
    $count = $this->cartModel->getCartCount($_SESSION['user_id']);

    $this->json(['success' => $result, 'cart_count' => $count]);
}

// ===== CART COUNT =====
public function getCountAjax() {
    $count = $this->cartModel->getCartCount($_SESSION['user_id']);
    $this->json(['cart_count' => $count]);
}
}
```

Code Explanation

- `requireLogin()` → Only logged-in users can access cart
- `index()` → Shows cart items and calculates total price
- `addAjax()` → Adds medicine to cart (checks stock first)
- `updateAjax()` → Updates quantity and recalculates total
- `removeAjax()` → Removes item from cart
- `getCountAjax()` → Returns total cart item count
- `$_SESSION['user_id']` → Identifies current user
- `json()` → Sends response for AJAX requests

Screenshot:

The screenshot displays the 'MediShop' web application interface. At the top, there's a navigation bar with links for Home, Profile, Cart (with a notification icon), and Logout. The main content area is titled 'Shopping Cart' and features a table with the following items:

Medicine	Price	Quantity	Subtotal	Action
Amoxicillin 500mg Pfizer	₺840.00	3	₺2,520.00	Remove
Napa Extend 665mg Besimco Pharma	₺50.00	1	₺50.00	Remove
Saline Nasal Drop AO Limited	₺90.00	2	₺180.00	Remove
Paracetamol 650mg GSK	₺480.00	1	₺480.00	Remove

Below the table, the total amount is displayed as 'Total: ₺3,230.00'. A blue button labeled 'Proceed to Checkout' is located at the bottom right of the cart area.

Feature 5: Order & Checkout System

Description:

The OrderController handles the complete checkout and order processing system for logged-in users. It manages the checkout flow in multiple steps, including saving the shipping address, checking product stock, and processing payment. It creates an order, updates medicine stock, records payment details, and clears the user's cart after successful purchase. Finally, it displays the order confirmation (success page) with full order details.

Relevant Source Code

File: medical_shop/controller/OrderController

```
<?php
class OrderController extends BaseController {
    private $cartModel;
    private $orderModel;
    private $paymentModel;
    private $medicineModel;
    private $userModel;

    public function __construct() {
        $this->requireLogin(); // user must be logged in

        $this->cartModel = new Cart();
        $this->orderModel = new Order();
        $this->paymentModel = new Payment();
        $this->medicineModel = new Medicine();
        $this->userModel = new User();
    }

    // ===== CHECKOUT PROCESS =====
    public function checkout() {
        $cartItems = $this->cartModel->getUserCart($_SESSION['user_id']);

        if (empty($cartItems)) {
            $this->redirect('controller=cart&action=index');
        }

        $user = $this->userModel->findById($_SESSION['user_id']);

        $total = array_sum(array_map(
            fn($i) => $i['price'] * $i['quantity'],
            $cartItems
        ));

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            // STEP 1: ADDRESS
            if ($_POST['step'] === 'address') {
                $_SESSION['checkout_address'] = $_POST['shipping_address'];

                $this->view('invoice', [
                    'cartItems' => $cartItems,
                    'total' => $total,
                    'address' => $_SESSION['checkout_address']
                ]);
            }
        }
    }
}
```

```
// STEP 2: PAYMENT
elseif ($_POST['step'] === 'payment') {
    // Check stock availability
    foreach ($cartItems as $item) {
        if (!$this->medicineModel->checkStock($item['id'], $item['quantity'])) {
            $this->view('checkout', [
                'cartItems' => $cartItems,
                'total' => $total,
                'user' => $user,
                'error' => "Insufficient stock for {$item['name']}"
            ]);
            return;
        }
    }

    // Create order
    $orderId = $this->orderModel->create(
        $_SESSION['user_id'],
        $total,
        $_SESSION['checkout_address'],
        $_POST['payment_method'],
        $cartItems
    );

    // Update stock
    foreach ($cartItems as $item) {
        $this->medicineModel->updateStock($item['id'], $item['quantity']);
    }

    // Create payment
    $this->paymentModel->create(
        $orderId,
        $total,
        $_POST['payment_method'],
        'TXN_' . time() . '-' . $_SESSION['user_id'] . $orderId
    );

    // Clear cart
    $this->cartModel->clearCart($_SESSION['user_id']);

    unset($_SESSION['checkout_address']);

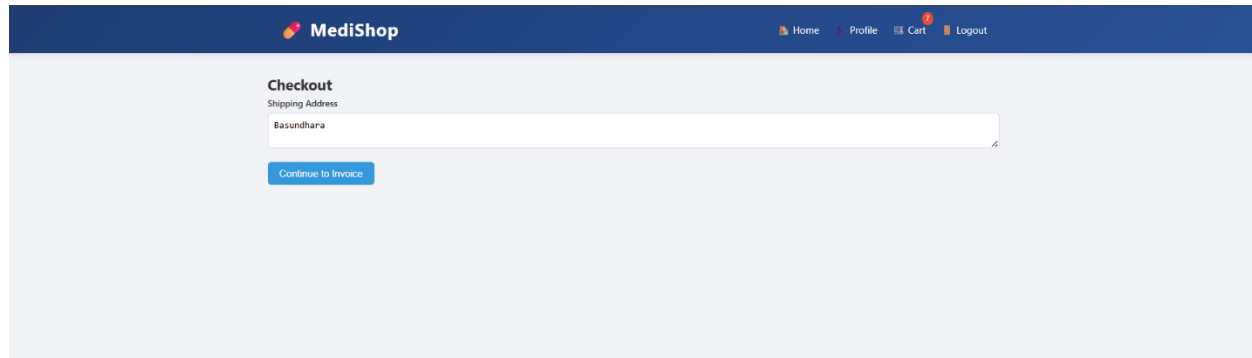
    $order = $this->orderModel->getOrderWithItems($orderId);

    $this->view('success', ['order' => $order]);
}

// SHOW CHECKOUT PAGE
else {
    $this->view('checkout', [
        'cartItems' => $cartItems,
        'total' => $total,
        'user' => $user
    ]);
}
}
```

Code Explanation

- `requireLogin()` → Only logged-in users can checkout
- `checkout()` → Handles full order process (address + payment)
- `getUserCart()` → Gets all cart items for user
- `$_POST['step']` → Controls checkout steps (address/payment)
- Stock check → Ensures medicine is available before order
- `orderModel->create()` → Creates new order record
- `medicineModel->updateStock()` → Reduces product stock
- `paymentModel->create()` → Saves payment transaction
- `clearCart()` → Empties user cart after order
- session → Stores shipping address temporarily
- success view → Shows final order confirmation



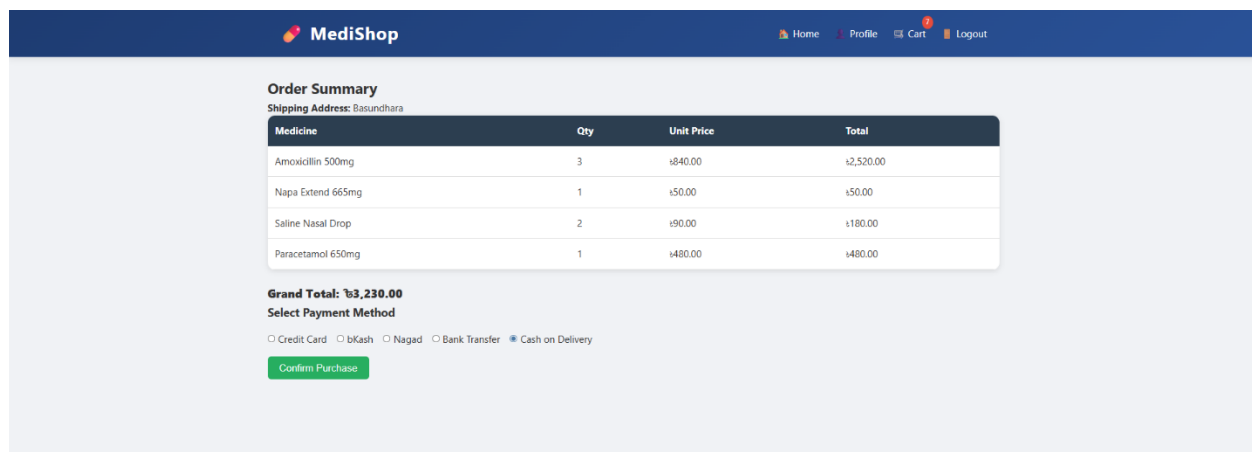
MediShop Home Profile Cart Logout

Checkout

Shipping Address

Basundhara

[Continue to Invoice](#)



MediShop Home Profile Cart Logout

Order Summary

Shipping Address: Basundhara

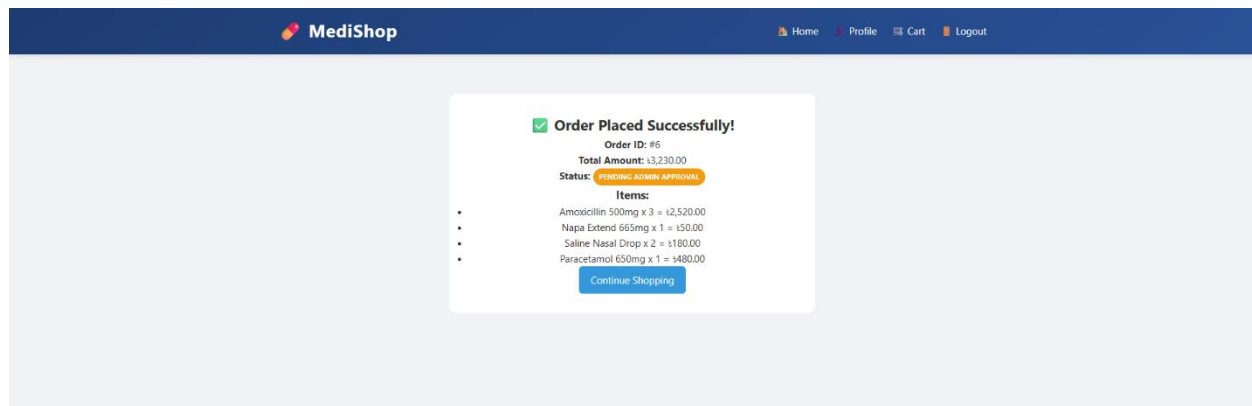
Medicine	Qty	Unit Price	Total
Amoxicillin 500mg	3	₹840.00	₹2,520.00
Napa Extend 665mg	1	₹50.00	₹50.00
Saline Nasal Drop	2	₹90.00	₹180.00
Paracetamol 650mg	1	₹480.00	₹480.00

Grand Total: ₹3,230.00

Select Payment Method

☐ Credit Card
 ☐ bKash
 ☐ Nagad
 ☐ Bank Transfer
 ☒ Cash on Delivery

[Confirm Purchase](#)



MediShop Home Profile Cart Logout

✓ **Order Placed Successfully!**

Order ID: #6

Total Amount: ₹3,230.00

Status: PENDING ADMIN APPROVAL

Items:

- Amoxicillin 500mg x 3 = ₹2,520.00
- Napa Extend 665mg x 1 = ₹50.00
- Saline Nasal Drop x 2 = ₹180.00
- Paracetamol 650mg x 1 = ₹480.00

[Continue Shopping](#)

Feature 6: Dashboard (Admin Panel)

Description:

The Admin Dashboard provides a quick overview of the system's key data, including total medicines, categories, customers, and pending orders. It displays these statistics in a card-based layout for easy monitoring. The dashboard also includes navigation links that allow the admin to manage customers, view purchase requests, and check order history efficiently.

Relevant Source Code

File: medical_shop/view/Dashboard.php

Code

```

<?php
// ===== ADMIN DASHBOARD VIEW =====
?>

<h2>Admin Dashboard</h2>

<!-- ===== STATISTICS SECTION ===== -->
<div class="stats-grid">

    <div class="stat-card">
        Medicines <br>
        <strong><?= $stats['medicines'] ?></strong>
    </div>

    <div class="stat-card">
        Categories <br>
        <strong><?= $stats['categories'] ?></strong>
    </div>

    <div class="stat-card">
        Customers <br>
        <strong><?= $stats['customers'] ?></strong>
    </div>

    <div class="stat-card">
        Pending Orders <br>
        <strong><?= $stats['pending_orders'] ?></strong>
    </div>

</div>

<!-- ===== ADMIN ACTION LINKS ===== -->
<div class="admin-links">

    <a href="index.php?controller=admin&action=customers" class="btn">
        Manage Customers
    </a>

    <a href="index.php?controller=admin&action=purchaseRequests" class="btn">
        Purchase Requests
    </a>

    <a href="index.php?controller=admin&action=history" class="btn">
        Order History
    </a>

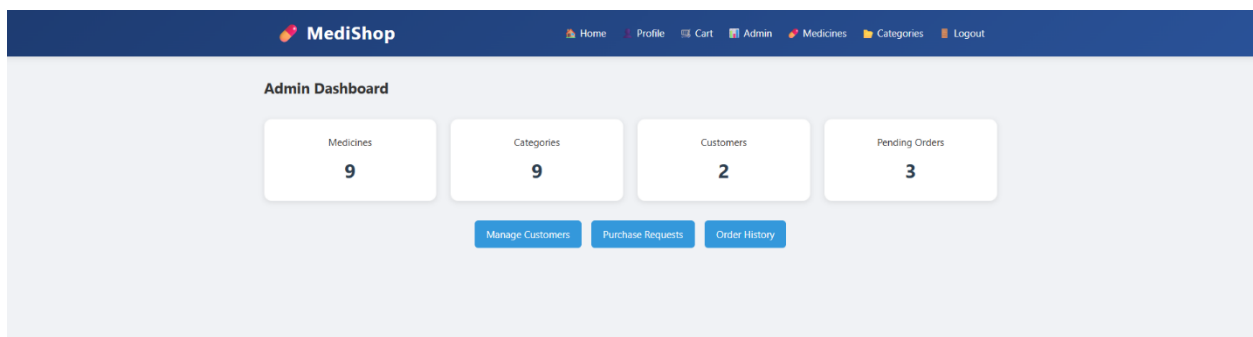
</div>

```

Explanation

- Displays dashboard heading
- Shows system statistics (medicines, categories, customers, pending orders)
- Uses \$stats array to fetch dynamic data
- stats-grid → Organizes cards in grid layout
- stat-card → Displays each count clearly
- → Highlights numbers
- <a href> → Redirects to admin pages
- Links → Customers, Purchase Requests, Order History

Screenshot:



Chapter 5: Conclusion & Future Work

5.1 Summary

This project developed a complete **online medicine management system** using the MVC architecture. The system includes features such as user authentication, medicine and category management, shopping cart, and order processing with payment handling. Admin users can manage medicines, categories, customers, and orders, while customers can browse medicines, add items to cart, and complete purchases.

The main goal of building a **secure, user-friendly, and efficient system** was successfully achieved. Throughout the development process, challenges such as handling session management, image uploads, and real-time cart updates were encountered and resolved through proper validation, structured coding, and use of AJAX.

5.2 Challenges Faced

- **Session Management Issues** → Managing login sessions and “Remember Me” functionality required careful handling of cookies and security.
- **Image Upload Handling** → Ensuring correct file type, size validation, and unique file naming was challenging.
- **Cart & AJAX Integration** → Implementing real-time cart updates without page reload required proper AJAX handling and JSON responses.
- **Stock Management** → Preventing users from ordering unavailable medicines required additional validation logic.
- **MVC Structure Understanding** → Organizing controllers, models, and views properly took time but improved code maintainability.

5.3 Future Improvements

- • Add **online payment gateway integration** (bKash, Nagad, card payment)
- • Implement **search and filter functionality** for medicines
- • Add **order tracking system** for customers
- • Improve UI/UX with modern responsive design
- • Add **email/SMS notification system** for order confirmation

References (APA Format)

- [1] PHP Documentation. (2024). *PHP Manual*. <https://www.php.net/docs.php>
- [2] Mozilla Developer Network. (2024). *HTTP & Web Development Guide*. <https://developer.mozilla.org>
- [3] W3Schools. (2024). *PHP Tutorial*. <https://www.w3schools.com/php/>
- [4] MySQL Documentation. (2024). *MySQL Reference Manual*. <https://dev.mysql.com/doc/>
- [5] jQuery Foundation. (2024). *jQuery Documentation*. <https://api.jquery.com/>